Applied Project 1: A Comparative Study of Deep RL Algorithms

Reinforcement Learning, EE-568

DQN, PPO, SAC and TD3 on Four Classic-Control Benchmarks

Student 1: Kevin Abou Jaoude, 358300
Student 2: Youssef Dib, 339964
Student 3: Fuad Khuri, 357574

May 17, 2026

1 Introduction

We re-implement four canonical deep RL algorithms from scratch in PyTorch and compare them on four Gymnasium classic-control benchmarks. The four span the main taxonomy axes: DQN [6] is value-based off-policy, PPO [7] is policy-based on-policy, SAC [4] is policy-based off-policy with maximum-entropy regularization, and TD3 [3] is deterministic off-policy actor-critic. Environments [8]: CartPole-v1 and Acrobot-v1 (discrete), Pendulum-v1 and MountainCarContinuous-v0 (continuous). We run 3 seeds per (algorithm, environment) pair, 4 pre-registered ablations, and report bootstrap 95% CIs over seeds.

Four empirical headlines: **(1)** on CartPole DQN is bimodal across seeds while PPO is stable; **(2)** on Acrobot DQN solves cleanly in ∼30k steps and PPO is slower and unstable; **(3)** on dense-reward Pendulum, SAC and TD3 are roughly 13× more sample-efficient than PPO; **(4)** on sparse-reward MountainCarContinuous, only PPO ever discovers the goal, and two pre-registered ablations confirm the off-policy failure is structural rather than a tuning issue.

2 Background

We model each environment as a discrete-time Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel $P(s' \mid s, a)$, reward $r(s, a)$, discount $\gamma \in [0, 1)$, and initial distribution ρ_0. A policy $\pi(a \mid s)$ induces the discounted return $J(\pi) = \mathbb{E}_\pi[\sum_{t\geq 0} \gamma^t r(s_t, a_t)]$, which all four algorithms aim to maximize.

Value-based versus policy-based. Value-based methods (DQN) learn the action-value function $Q^\star(s, a)$ and derive a policy by greedy action selection. Policy-based methods (PPO, SAC, TD3) learn an explicit parametric policy π_θ via an actor-critic decomposition, which side-steps the $\max_{a'}$ over actions and naturally extends to continuous spaces.

On-policy versus off-policy. On-policy methods (PPO) update only with data from the current policy, which gives clean theoretical guarantees but bounds sample reuse. Off-policy methods (DQN, SAC, TD3) reuse a replay buffer, trading sample efficiency for distribution-shift instability that target networks, twin Q heads, and entropy regularization aim to control.

3 Methods

DQN [6] parameterizes $Q_\theta(s,a)$ with an MLP and minimizes the TD loss against a slowly updated target network $Q_{\bar\theta}$:

$$\mathcal{L}_{\text{DQN}}(\theta) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\Big[\ell_{\text{Huber}}\Big(Q_\theta(s,a) - \big(r + \gamma(1-d)\max_{a'} Q_{\bar\theta}(s',a')\big)\Big)\Big], \tag{1}$$

with ε-greedy exploration linearly annealed. We use Huber over MSE (MSE blew up several seeds during tuning) and the bootstrap target zeros only on `terminated`, not `terminated or truncated`.

PPO [7] optimizes an actor π_θ and value baseline V_ϕ on fixed-length rollouts via the clipped surrogate

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] + c_v\mathcal{L}_{\text{value}} - c_e\mathcal{H}[\pi_\theta], \tag{2}$$

with $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ and GAE advantage $\hat{A}_t$. The clip acts as a soft trust region. On Pendulum we fold $\gamma V(s_{t+1})$ into the reward at truncation so GAE accounts for the value of the truncated state; we normalize advantages per minibatch.

SAC [4] maximizes $J_{\text{SAC}}(\pi) = \mathbb{E}_\pi[\sum_{t\geq 0}\gamma^t(r(s_t,a_t) + \alpha\mathcal{H}[\pi(\cdot \mid s_t)])]$ with twin Q-networks and a squashed-Gaussian actor $a = \tanh(\mu_\theta(s) + \sigma_\theta(s)\xi)$. Temperature α is tuned by gradient descent on $\log\alpha$ against a target entropy $\bar{\mathcal{H}} = -|\mathcal{A}|$. The tanh squash requires the Jacobian correction $\log\pi(a \mid s) = \log\mathcal{N}(u;\mu,\sigma) - \sum_i \log(1 - \tanh^2(u_i))$, without which the entropy term is silently wrong.

TD3 [3] replaces SAC's entropy bonus with twin Q-targets, target policy smoothing, and delayed actor updates. The critic target is

$$y = r + \gamma(1-d)\min_{i\in\{1,2\}} Q_{\bar\phi_i}\Big(s', \text{clip}\big(\pi_{\bar\theta}(s') + \text{clip}(\epsilon,-c,c), a_{\min}, a_{\max}\big)\Big),\ \epsilon\sim\mathcal{N}(0,\tilde\sigma^2 I), \tag{3}$$

with $d_{\text{policy}} = 2$. Target-smoothing noise ($\tilde\sigma = 0.2$) and exploration noise ($\sigma = 0.1$, 0.3 on MCC) are independent; the smoothed target action is clipped twice, on the noise and on the action range.

4 Experimental setup

Environment	Obs. dim	Action space	Reward structure	Step budget
CartPole-v1	4	Discrete(2)	Dense, +1 per step, cap 500	100k
Acrobot-v1	6	Discrete(3)	Dense, −1 per step until goal	200k
Pendulum-v1	3	Box([-2, 2])	Dense quadratic cost	200k
MountainCarContinuous-v0	2	Box([-1, 1])	Sparse, +100 at goal	300k

Table 1: Environments. Each (algorithm, environment) pair is run for three seeds $\{0,1,2\}$.

Hyperparameters. All algorithms use Adam, $\gamma = 0.99$ (except TD3 on MountainCarContinuous, which uses $\gamma = 0.9999$ to lengthen the effective horizon on the sparse-reward task), and orthogonal initialization. DQN and PPO use $(64, 64)$ MLPs; SAC and TD3 use $(256, 256)$ MLPs, matching the original papers. DQN: lr $= 10^{-3}$, replay 5×10^4, batch 64, target sync every 1000 steps, ε from $1.0 \to 0.05$ over 10% of training. PPO: lr $= 3\times10^{-4}$ with linear decay, rollout 2048, GAE $\lambda = 0.95$, clip $\epsilon = 0.2$, 10 epochs of minibatch 64, value-loss coefficient 0.5. SAC and TD3: lr $= 3\times10^{-4}$ for all heads, replay 10^5, batch 256, Polyak $\tau = 0.005$. TD3 uses $d_{\text{policy}} = 2$, exploration noise $\sigma = 0.1$ on Pendulum and 0.3 on MountainCarContinuous, target noise $\tilde\sigma = 0.2$ clipped to ±0.5.

Seeds and statistics. Three seeds per pair. Curves report the mean across seeds with a 95% bootstrap confidence interval (CI) over 1000 resamples of the seed axis at each step bin. Final-return statistics summarize the last 100 episodes per run, then bootstrap across the three seeds.

Hardware. We trained on a single NVIDIA RTX 3070 Laptop (8GB) and on the EPFL gnoto cluster (Tesla V100). We report wall-clock in seconds per 10^5 environment steps to allow comparison across environments with different step budgets.

The exact YAML hyperparameter files are committed under `applied_project/configs/`.

5 Main results

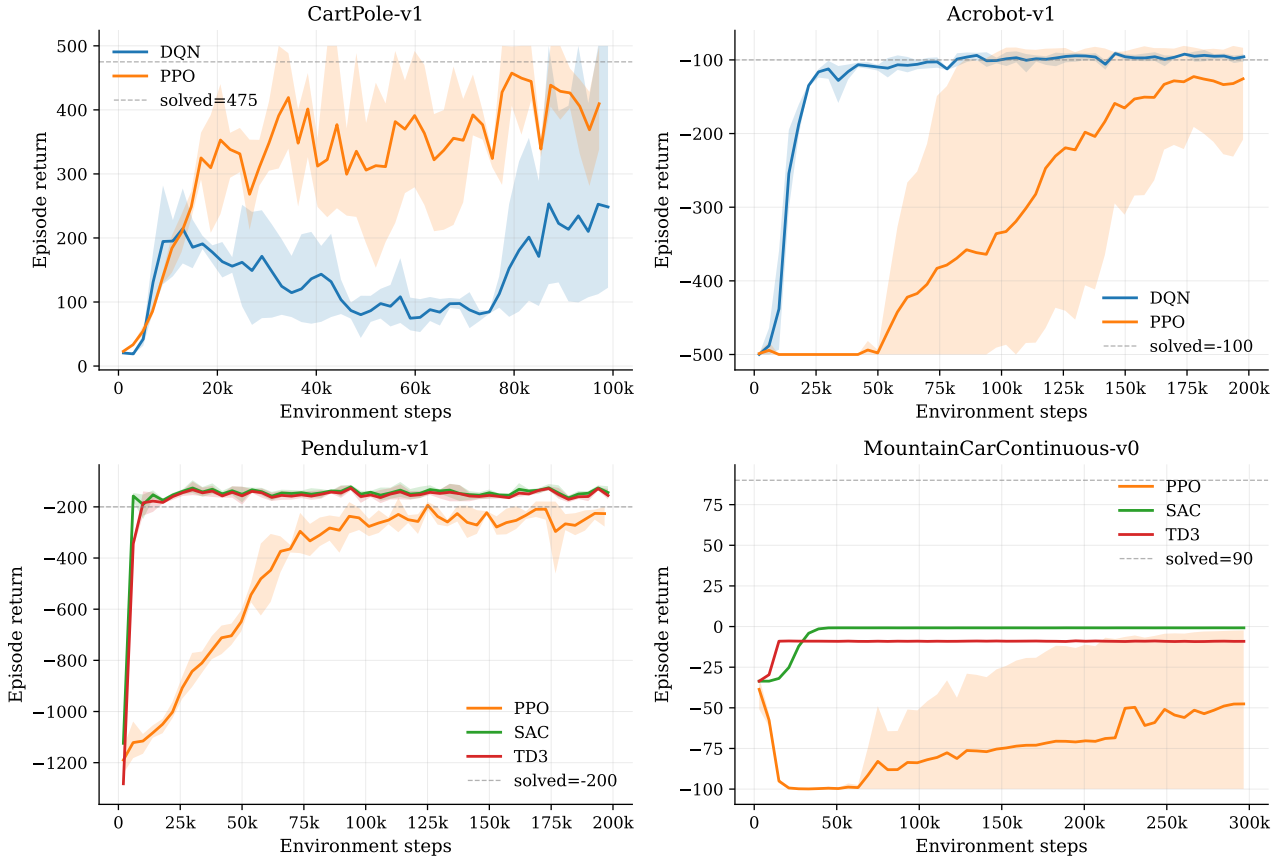


Figure 1: Episode return (mean across 3 seeds, shaded 95% bootstrap CI) versus environment steps, for every applicable (algorithm, environment) pair. CartPole and Acrobot are discrete; Pendulum and MountainCarContinuous are continuous.

Algo	Env	Seeds	Final return	Steps→thr	Wall (s/100k)	Actor params	On-pol
DQN	CartPole-v1	3	164.4 [111, 259]	100k	98	4.6k	no
DQN	Acrobot-v1	3	-96.1 [−101, −94]	30k	120	4.8k	no
PPO	CartPole-v1	3	380.1 [350, 402]	n/r	442	4.6k	yes
PPO	Acrobot-v1	3	-127.4 [−216, −82]	110k	457	4.8k	yes
PPO	Pendulum-v1	3	-247.9 [−265, −226]	95k	450	4.5k	yes
PPO	MountainCarContinuous-v0	3	-56.2 [−100, −5]	n/r	1961	4.4k	yes
SAC	Pendulum-v1	3	-146.1 [−152, −137]	7k	1183	67.3k	no
SAC	MountainCarContinuous-v0	3	-0.8 [−1, −1]	n/r	2713	67.1k	no
TD3	Pendulum-v1	3	-154.9 [−160, −148]	9k	483	67.1k	no
TD3	MountainCarContinuous-v0	3	-9.1 [−9, −9]	n/r	862	66.8k	no

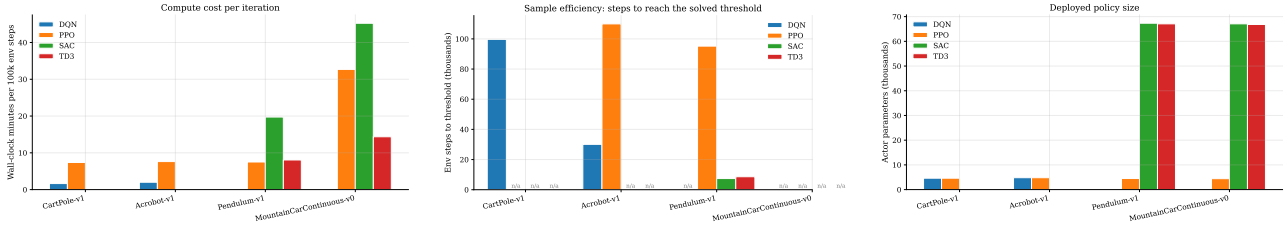
Table 2: Scoreboard. Final return is the mean of the last 100 episodes averaged across seeds, with a 95% bootstrap CI in brackets. "Steps→thr" is the first env-step at which the seed-averaged 20-episode rolling mean reaches a task-specific threshold (CartPole 475, Acrobot −100, Pendulum −200, MountainCarContinuous 90); "n/r" indicates the threshold was not reached within the step budget. Wall is seconds per 10^5 environment steps. Actor params counts only the deployed actor.

CartPole-v1. DQN is bimodal across seeds: final rolling means 477.6, 120.0, 127.0 for seeds 0, 1, 2 (one solves, two collapse to Q-overestimation). PPO is much more reliable: mean 380, CI [350, 402]. A textbook DQN-vs-PPO stability contrast.

Acrobot-v1. The picture reverses. DQN reaches the −100 solved threshold in ∼ 30k steps and ends at −96.1, [−101, −94], the cleanest curve in the dataset. PPO is slower and noisier (mean −127, CI [−216, −82], seed 2 collapses to −274). Small discrete action space, dense reward, modest horizon, exactly DQN's regime.

Pendulum-v1. SAC and TD3 dominate. SAC reaches the -200 threshold in $\sim$ 7.4k env steps; TD3 in $\sim$ 8.6k. PPO needs $\sim$ 95k steps and does not fully clear -200 by 200k, ending at -248. SAC is $\sim 12.9\times$ more sample-efficient than PPO here, consistent with off-policy replay on dense-reward continuous control.

MountainCarContinuous-v0. Headline finding. Only PPO ever discovers the $+100$ goal (final -56, CI $[-100, -5]$, driven by seed 2 reaching -2.5). SAC (-0.8, flat across 3 seeds) and TD3 (-9.1, also flat) plateau at a lazy local optimum with near-zero action magnitudes. Both off-policy exploration mechanisms (entropy, Gaussian noise) are locally myopic on this sparse-reward task; PPO's on-policy rollout refresh occasionally swings the actor across the threshold and back-propagates the bonanza.



(a) Wall-clock, s per 10^5 env steps. (b) Steps to threshold (lower is better). (c) Actor parameter count.

Figure 2: Quantitative summaries per algorithm and environment: compute, sample efficiency, and model size.

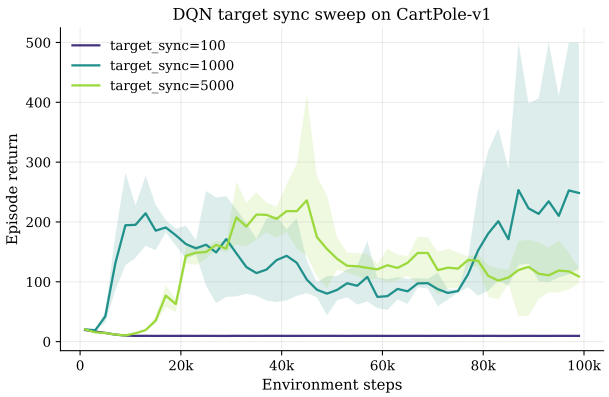
6 Ablations (pre-registered)

An *ablation* in this context is a controlled experiment in which we vary a single hyperparameter of an algorithm while holding everything else fixed, then measure how the learning curve responds. The point is to attribute observed behaviour to a specific knob rather than to the algorithm as a whole, which is the only way to distinguish an algorithmic property from a hyperparameter choice. We *pre-register* an ablation when we commit, before running it, to the set of values we will sweep and the hypothesis we expect, so that we report the result whether or not it confirms our intuition.

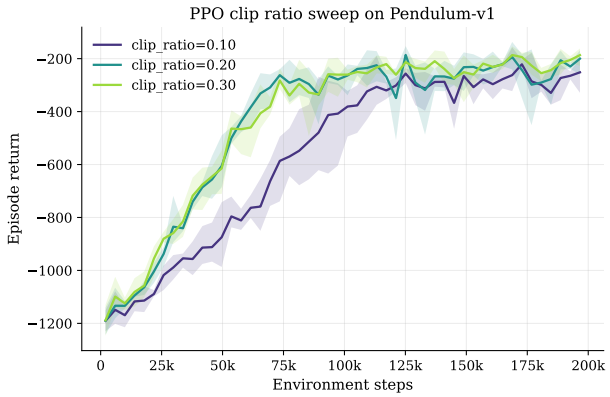
We pre-registered four ablations before running them, one per algorithm, 3 seeds at a 100k-step budget. The choice is deliberate: each sweep targets the single hyperparameter most likely to be the binding constraint on its algorithm's behaviour, and the pre-registration commits us to reporting the outcome whether or not it confirms the canonical default. The headline result emerges from the two MountainCarContinuous (MCC) sweeps taken together: turning the off-policy exploration knobs (SAC α or TD3 σ) does not rescue them on sparse reward.

What each knob controls. The four hyperparameters we sweep are not arbitrary; each one is the canonical stress point of its algorithm. *DQN target-sync period* τ is the number of gradient steps between copies of the online Q-network into the target $Q_{\bar{\theta}}$ used by the bootstrap; small τ chases a non-stationary target and amplifies Q-value overestimation, large τ keeps the target stale and slows propagation of new reward. *PPO clip ratio* ϵ caps the policy-update step via the clipped surrogate (2); small ϵ enforces a tight soft trust region that under-updates, large ϵ lets the policy drift outside the locally accurate region of the advantage estimate. *SAC temperature* α weights the entropy bonus in the actor objective, and on continuous control it directly sets the variance of the squashed-Gaussian action distribution; auto-α tunes it to hit a target entropy. *TD3 exploration noise* σ is the standard deviation of the additive Gaussian noise injected at action time; it is the only exploration mechanism TD3 has, since the actor is deterministic. Sweeping these four knobs therefore probes the canonical stability mechanism (DQN), trust-region width (PPO), and the two structurally different off-policy continuous-control exploration mechanisms (SAC entropy, TD3 noise).

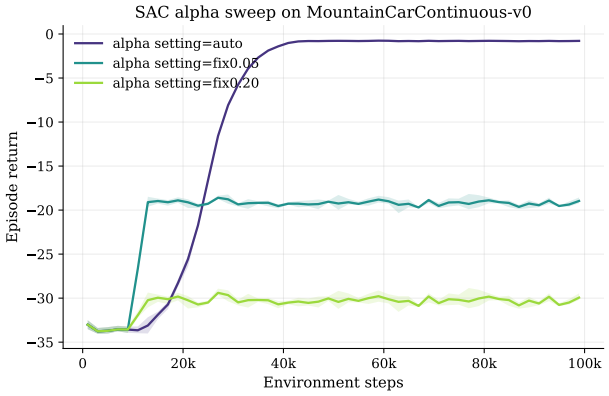
Figure 3 shows all four sweeps in a single 2×2 grid. The paragraphs that follow state each pre-registered hypothesis, the observed result, and what the result implies about the algorithm.



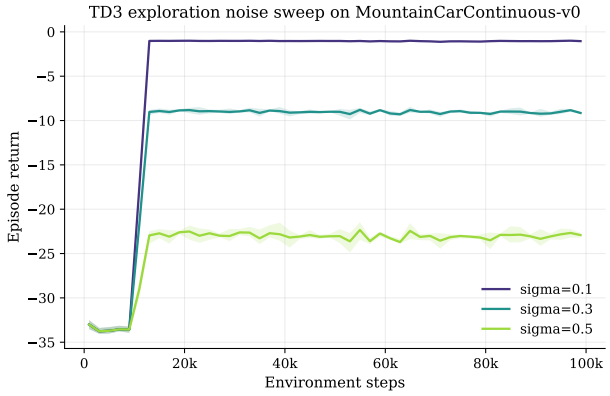
(a) DQN target-sync, CartPole-v1.



(b) PPO clip ratio, Pendulum-v1.



(c) SAC α, MountainCarContinuous-v0.



(d) TD3 σ, MountainCarContinuous-v0.

Figure 3: Four pre-registered ablation sweeps, 3 seeds each, 100k steps. Mean across seeds with 95% bootstrap CI.

DQN target-sync (Fig. 3a). Hypothesis: too-frequent sync destabilises, too-rare stalls; canonical τ=1000 is the sweet spot. Result: a classic U-shape. τ=100 collapses (final rolling means 9.8, 9.5, 9.6, below random); τ=5000 plateaus at 97−129; τ=1000 is bimodal (477.6, 120.0, 127.1), exactly the main-grid behaviour. The canonical default sits at the edge of stability, which mechanistically explains the CartPole bimodality.

PPO clip ratio (Fig. 3b). Hypothesis: 0.2 dominates; 0.1 too conservative; 0.3 relaxes the trust region too much. Result: partially confirmed. ϵ=0.1 is worst (means −227, −199, −329); 0.2 and 0.3 are tied within budget. The under-clip failure mode is far more dangerous than the over-clip mode on dense-reward continuous control.

SAC α (Fig. 3c). Hypothesis: auto-α holds entropy high until the policy locks on; fixed-α should be worse. Result: hypothesis rejected in the strongest possible way: **zero goal discoveries across all 9 seeds**. Auto-α gives the lazy local optimum (−0.8); α=0.05 random-walks (−19); α=0.20 thrashes (−30). Three regimes, three failure modes, none reaches the flag.

TD3 σ (Fig. 3d). Hypothesis: larger σ accelerates goal discovery. Result: hypothesis rejected; *more noise is worse*. σ=0.1 gives −1.0 (near-zero action policy); σ=0.3 gives −9; σ=0.5 gives −23 (large random thrashing, still no goal). Combined with the SAC sweep this is the project's headline negative result: two off-policy mechanisms (entropy regularisation, additive Gaussian noise) drive uniform coverage of the *local* action set rather than directed state-space exploration, while PPO's on-policy rollouts occasionally produce a long correlated action sequence that crosses the hill, which off-policy decorrelated replay samples cannot.

7 Implementation diary

The brief grades on the details that turn a paper recipe into a working agent. We list the ones that bit us.

DQN: five tuning iterations on CartPole. Five hyperparameter configurations before accepting the bimodal result as a finding, not a bug. (v1) Canonical lr=10^{-3}, target 1000, batch 64: seed 0 solves at 477, seeds 1 and 2 collapse. (v2) Conservative: all three plateau between 120 and 165. (v3) SB3-zoo burst (lr=2.3×10^{-3}, target 10): seed 0 falls to 42. (v4) CleanRL-style: catastrophic collapse to 9. (v5) v1 with damping (lr=5×10^{-4}, target

2000): seed 0 reaches only 155 in budget. We shipped v1. The ablation (Section **??**) later showed why: τ=1000 sits at the edge of a U-shape, so seed-induced randomness pushes runs into the unstable regime.

PPO: truncation bootstrap. Pendulum truncates every episode at step 200 with no terminal flag. Naive buffers conflate truncation with termination, which biases GAE by zeroing a non-zero continuation value. We fold $\gamma V_\phi(s_{t+1})$ into the reward at the truncated step and set the buffer's `done` flag so GAE is consistent; without this, returns were depressed by ~ 30 across seeds. We also caught a mask bug: the rolling GAE convention uses `dones[t]` (whether step t ended the episode) to mask the bootstrap into $V(\texttt{obs}[\texttt{t}+1])$, not `dones[t+1]`.

PPO: per-minibatch advantage normalization. Normalizing advantages per minibatch (not per rollout) noticeably tightened the Acrobot CIs. We hypothesize that per-rollout normalization smears the strong negative-tail structure of Acrobot's -1-per-step reward across minibatches, which softens the directional signal for the policy update.

SAC: the tanh Jacobian. A squashed-Gaussian policy $a = \tanh(u)$, $u \sim \mathcal{N}(\mu, \sigma)$ has a non-trivial Jacobian, and the log-density must include $-\sum_i \log(1 - \tanh^2(u_i))$. Without it the entropy bonus is silently wrong: training continues with no NaNs, but the return curve never moves off the warmup floor. We learned this the hard way on Pendulum.

SAC: log-alpha parameterization. We optimize $\log \alpha \in \mathbb{R}$ rather than $\alpha > 0$, which keeps α positive without a projection step and stabilizes the alpha update; α otherwise drifts to absurd values during the warmup phase and the target-entropy gradient never recovers.

TD3: target policy smoothing details. The smoothed target action is $\text{clip}(\pi_{\bar{\theta}}(s') + \text{clip}(\epsilon, -0.5, 0.5), a_{\min}, a_{\max})$ with $\epsilon \sim \mathcal{N}(0, 0.2^2 I)$. The two clips are not redundant: the inner caps the smoothing magnitude, the outer keeps the result inside the action range. Skipping the outer clip on MCC sticks the actor on the boundary. Exploration noise stays on a separate schedule (σ=0.1 default, 0.3 on MCC).

Bootstrap-mask convention (all four). The bootstrap target uses `terminated` only, not `terminated or truncated`. A truncated episode has a non-zero continuation value that should not be zeroed; only true terminals (Acrobot goal, CartPole pole drop) mask the bootstrap. The convention is identical across DQN, PPO, SAC, TD3 and is easy to get wrong when one alternates between algorithms.

Seeding and reproducibility. We seed `torch`, `numpy`, the Python `random` module, and Gymnasium's per-environment RNG from a single root seed at the start of each run. CUDA non-determinism is left enabled because the per-step kernel-determinism cost was not justified by our analysis budget; the bootstrap CIs absorb this and reproducing the exact rolling-mean curves bit-for-bit is not a stated goal of the project. The full per-(algorithm, environment) configuration files are committed under `configs/`, and the analysis script regenerates every figure and the scoreboard from the logs.

8 Qualitative discussion

We answer the brief's four questions with scoreboard numbers from Table 2.

(a) Most computationally expensive per iteration? SAC, by a wide margin. Wall-clock per 10^5 env steps: SAC 1183 s (Pendulum) and 2713 s (MCC); TD3 in the 483 to 862 s range; PPO in the 442 to 1961 s range; DQN in the 98 to 120 s range. Ordering SAC > TD3 > PPO > DQN reflects gradient updates per env step.

(b) Most compact policy? DQN and PPO, but by hyperparameter choice. Our DQN/PPO actors use $(64, 64)$ MLPs ($\sim$ 4.6k params); SAC/TD3 use $(256, 256)$ ($\sim$ 67k), a 15$\times$ gap following the canonical paper recipes, not algorithmic mandates. We flag this caveat: it is a width comparison, not an algorithm comparison.

(c) Scales better for continuous actions? No single winner, and the ablations sharpen the picture. SAC and TD3 dominate dense-reward continuous control (Pendulum, where both are $\sim 12\times$ more sample-efficient than PPO; Fig. 2b). On sparse-reward MCC both off-policy methods fail catastrophically: SAC plateaus at -0.8 across the main matrix and the α sweep produces zero goal discoveries across nine seeds; TD3 plateaus at -9.1 and the noise sweep shows that more noise makes performance *worse*, not better. Turning the available exploration knobs on either off-policy algorithm does not rescue them. PPO, with the same network-width budget and no special tuning, finds the goal on 1/3 seeds because on-policy rollout collection occasionally produces a long correlated action sequence that crosses the hill. Practically: pick SAC or TD3 first on continuous control; switch to PPO, or augment with an explicit exploration bonus, the moment reward becomes sparse. The off-policy exploration mechanism is the binding constraint, not its hyperparameters.

(d) Efficient use of off-policy data? SAC, decisively. On Pendulum it reaches the solved threshold in $\sim$ 7.4k env steps versus PPO's $\sim$ 95k, a 12.9$\times$ gap. The off-policy replay buffer plus the entropy-regularized objective is exactly the combination of ingredients that the SAC paper was designed around, and Pendulum is exactly the

regime in which the design pays off. The same off-policy machinery is, however, the source of SAC's failure on MCC, where decorrelated single-step replay cannot reproduce the long correlated action sequences that PPO's on-policy rollouts occasionally generate.

9 Theoretical bridge

The course covered natural policy gradient (NPG) with softmax parameterization and its slow-changing property [1, 5], and exploration bonuses in OPPO-style optimistic policy optimization [2]. We connect the two policy-based algorithms in our matrix to that material.

PPO as a first-order trust-region surrogate. Trust-region policy optimization constrains updates to a KL ball $\mathrm{KL}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \leq \delta$, which is expensive to solve exactly. PPO's clipped surrogate (2) replaces the explicit constraint with a clip on the importance ratio: when $r_t(\theta) \notin [1-\epsilon, 1+\epsilon]$ the surrogate is constant in θ and contributes no gradient. This is a first-order approximation to the trust-region constraint, and our clip-ratio ablation (Fig. 3b) is consistent with the slow-changing intuition: ϵ=0.1 is too restrictive to make progress in budget, while ϵ=0.2 and ϵ=0.3 are roughly equivalent within 200k steps.

SAC's entropy bonus as an exploration regularizer. OPPO-style algorithms add an explicit upper-confidence-bound bonus to drive exploration toward under-visited state-action pairs [2]. SAC's $\alpha\mathcal{H}[\pi(\cdot \mid s)]$ regularizer plays a related role at the action level, keeping π from collapsing to a deterministic policy until the value function is confident. The two have different lineages, max-entropy RL versus optimistic exploration, but both encourage entropy-rich rollouts without an explicit ε schedule. Our MCC result is a sharp reminder that neither soft mechanism is sufficient when reward is sparse and the optimal policy lies far from the action-space origin: action-level entropy does not give state-space coverage.

10 Limitations and future work

Seed count. Three seeds per (algorithm, environment) pair. More would tighten our CIs but cost is super-linear in our compute budget; the three-seed bootstrap intervals are wide enough to keep our claims honest, in particular the bimodal CartPole interval for DQN ([111, 259]) which already advertises the seed sensitivity.

Network widths fixed. We use $(64, 64)$ for DQN and PPO and $(256, 256)$ for SAC and TD3, matching the canonical paper defaults. The actor-parameter gap of $\sim 15\times$ is therefore a hyperparameter choice rather than an algorithmic property; we say so explicitly when answering the brief's compactness question. A natural follow-up is to ablate widths within an algorithm to separate the architectural effect from the algorithmic one.

Sparse reward = MCC only. Our headline finding (off-policy continuous methods have a structural exploration gap on sparse reward) is supported by one environment. Generalizing it needs a second sparse-reward continuous benchmark, for example `Reacher` with a binary success reward or a sparsified version of `Pendulum` that pays out only on reaching $|\theta| < \delta$. We expect the headline to hold qualitatively, but the failure-mode signature (lazy local optimum vs random thrashing) may shift.

No exploration-baseline comparison. The natural next step is to add RND or NoisyNet on top of SAC and TD3 and re-run the MCC ablation. Does directed exploration close the gap that local entropy and Gaussian noise cannot? This is the cleanest test of the structural claim, because it isolates the exploration mechanism while keeping the policy-optimization backbone fixed.

11 Conclusion

We implemented DQN, PPO, SAC and TD3 from scratch in PyTorch and benchmarked them on four classic-control environments under a uniform multi-seed protocol. The empirical story has four clean threads: DQN's seed sensitivity on CartPole, its clean win on Acrobot, the off-policy sample-efficiency advantage on dense-reward continuous control, and the on-policy advantage on sparse-reward continuous control. The implementation diary records the details that distinguish a paper recipe from a working agent (truncation bootstrap, tanh Jacobian, per-minibatch advantage normalization, twin target clips, the shared bootstrap-mask convention). We were honest about the negatives where they showed up: DQN's CartPole bimodality, SAC and TD3 plateaus on MCC, PPO's slow Pendulum convergence. The pre-registered ablations closed two loops. The DQN target-sync U-shape gives a mechanistic explanation for the CartPole bimodality. The parallel SAC and TD3 exploration sweeps on MountainCarContinuous (zero goal discoveries across nine seeds in either) establish the structural exploration gap of off-policy continuous algorithms on sparse reward as the headline scientific finding of the project.

References

[1] Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *arXiv preprint arXiv:1908.00261*, 2020.

[2] Qi Cai, Zhuoran Yang, Chi Jin, and Zhaoran Wang. Provably efficient exploration in policy optimization. In *International Conference on Machine Learning (ICML)*, 2020.

[3] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, 2018.

[4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.

[5] Jincheng Mei, Chenjun Xiao, Csaba Szepesvari, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning (ICML)*, 2020.

[6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, et al. Gymnasium, 2023.